

1. Rozwiązanie rzeczywistego problemu (podejście „od studentów dla studentów”)

Największą siłą naszej aplikacji jest fakt, że nie jest to kolejny sztuczny projekt stworzony wyłącznie „na zaliczenie”. Student Planner to odpowiedź na autentyczny, ludzki problem chaosu organizacyjnego podczas studiów. Jako studenci stanowimy jednocześnie grupę docelową naszego produktu. Posiadamy głęboką wiedzę domenową – nikt nie rozumie potrzeb, frustracji i wyzwań studenta w trakcie sesji tak dobrze, jak my sami. Dzięki temu każda funkcja w aplikacji ma praktyczne zastosowanie i realnie ułatwia życie.

2. Dbłość o inżynierię oprogramowania i zasady SOLID Cały system został zaprojektowany i napisany przez nas całkowicie od zera, bez polegania na gotowych szablonach. Podczas procesu wytwórczego kładliśmy ogromny nacisk na jakość architektury. Staraliśmy się konsekwentnie implementować zasady czystego kodu oraz SOLID. Dzięki podziałowi na moduły (osobna warstwa widoków Kivy, osobne repozytorium bazy danych, izolacja logiki), kod jest skalowalny, łatwy w utrzymaniu i gotowy na ewentualną dalszą rozbudowę.

3. Bezpieczeństwo i niezależność W erze aplikacji chmurowych postawiliśmy na świadomy krok w stronę prywatności i stabilności. Nasz system działa w oparciu o lokalną bazę danych SQLite, co oznacza, że jest w 100% niezależny od połączenia z Internetem. Aplikacja nie wysyła żadnych wrażliwych danych edukacyjnych na zewnętrzne serwery. Gwarantuje to absolutne bezpieczeństwo informacji użytkownika i całkowitą odporność na awarie infrastruktury uczelnianej.

4. Rozwój kompetencji zespołu i profesjonalny workflow Projekt ten wygenerował dla nas ogromną wartość edukacyjną, wykraczającą poza samo pisanie kodu w Pythonie. Było to dla nas pierwsze tak duże przedsięwzięcie, w którym wdrożyliśmy w pełni profesjonalny przepływ pracy oparty na systemie kontroli wersji na platformie GitHub. Nauczyliśmy się w praktyce zarządzania gałęziami, rozwiązywania konfliktów oraz asynchronicznej pracy zespołowej. Nabyte w ten sposób doświadczenia na pewno przygotowują nas do pracy w branży IT.

5. Stabilność potwierdzona automatyzacją Dzięki wdrożeniu testów automatycznych z użyciem frameworka pytest (mechanizm mockowania bazy danych, izolowane funkcje), udowodniliśmy, że nasz system jest odporny na błędy logiczne i niepoprawne dane wejściowe. Osiągnęliśmy dojrzałość projektu, która pozwala na bezpieczne użytkowanie aplikacji w środowisku produkcyjnym.